

FinVerify: Improving the Reliability of LLM-Based Financial Advisors Through Hybrid Verification Systems

Nikhil Sharma

UNI: ns3942

COMS 6156 — Final Report — April 2026

Abstract

Large language models are increasingly being used as personal financial advisors, especially among young adults and those with first-time jobs. LLMs also have significant risks of confidently presenting incorrect or harmful information (hallucinations), which poses real challenges in a field like personal finance management, where incorrect information can directly cost users money. This paper presents FinVerify, a retrieval-augmented system that grounds LLM-generated financial advice in a knowledge base of 18 authoritative documents from agencies such as the IRS, CFPB, SEC, and SSA. We evaluate FinVerify against an open-weighted baseline model (Qwen2.5-3B-Instruct) on a 156-question dataset covering six personal finance topics, including tax, investments, debt, and retirement. We also use real-life questions sampled from platforms like Reddit. FinVerify improves judge mean from 2.99 to 4.58 and MC accuracy from 76.7% to 96.7% over the baseline, while an ablation study shows that a lot of the accuracy gain is due to model scale rather than retrieval alone. However, FinVerify is the only model that provides verifiable inline citations (33% mean coverage, 106/156 answers with at least one citation), a property that’s not captured by raw accuracy metrics but is essential for real-world usage of systems like these in order for users to feel comfortable and verify the information they’re receiving. All code, data, and evaluation scripts are publicly available on GitHub.

1. Synopsis

The growth of large language models (LLMs) has already ushered in innovation and per-

sonalization across many industries, including medicine, education, and more. Similarly, LLMs and AI have the potential to revolutionize the digital personal financial advisor space. There is a wide variety of existing literature on various forms of digital personal financial advisors, from rule-based algorithms to robo-advisors to LLM-based tools. As a quick recap, rule-based financial tools use predefined decision logic and behavioral nudges, such as automated savings reminders and spending alerts, to reduce inertia and encourage better financial habits. Robo-advisors go further by using quantitative models and algorithmic rebalancing to automate portfolio management. These tools are primarily used to improve diversification and reduce volatility in users’ asset portfolios. LLM-based tools represent the newest generation, using conversational interactions and context retention to provide personalized financial guidance across a broader range of topics than either of the previous approaches. There is also work that concludes that LLM-based digital personal financial management tools represent a true paradigm shift and have the potential to revolutionize this space. However, there are also real concerns associated with these new LLM-based tools. LLMs are prone to hallucinations, which means that they confidently give incorrect information to users without indicating that they are unsure. This can lead users to trust all the information they receive without applying their own critical thinking. This can result in some sticky situations, especially in the personal finance space, where an LLM could advise poor financial decisions or recommend an investment in a poor area.

The system we created, FinVerify, aims to address the downside of LLM hallucinations by

grounding the LLM’s answer in verifiable, authoritative source material. These source materials, including documents from agencies such as the IRS, CFPB, SEC, SSA, and Health-Care.gov, form a knowledge base against which the FinVerify model is grounded. This type of architecture is inspired by existing literature and implementations of Retrieval-Augmented Generation (RAG) [1], which will be discussed in the next section. Using RAG, FinVerify works in a pipeline manner: (1) the user asks a personal-finance question, (2) the question is embedded and used to retrieve the top-k most relevant passages from a vector store of authoritative documents published by agencies and (3) the LLM generates its answer conditioned on the knowledge base vector store, with inline [n] citations pointing back to the chunks it relied on. The specific mechanisms, such as the embedding model, chunking strategy, retrieval index, and prompt design, are defined in the methodology section.

To test FinVerify and evaluate its output against SOTA models today, we compare the RAG-augmented model against a baseline model that answers solely based on its pre-trained knowledge, without any additional retrieval mechanisms. We score both systems using accuracy on multiple-choice questions from a dataset we put together, as well as cosine similarity against a reference answer. We use an LLM-as-judge approach to create a rubric that assesses factuality, completeness, and advice quality for both the baseline and the RAG-augmented model. For the latter system, we also track and report citation coverage (the fraction of retrieved passages the model actually cites) to identify cases where knowledge retrieval succeeded but was ignored in the output.

While we will fully explore the evaluation and metrics of the RAG-augmented system in a later section, the data show that adding the verification layer significantly boosts the accuracy and quality of responses in both open-ended and multiple-choice formats. The key innovation of this project is the hybrid combination of structured, rules-based verification with gener-

ative LLM output. This combines the best of both types of digital personal finance systems, and the direct empirical comparison with an LLM-only baseline shows fewer hallucinations. The FinVerify system, the question-and-answer dataset we compiled, and the evaluation framework are all publicly available on GitHub for others to use and build upon.

1.1 Digital Financial Advice Tools

The scope and capabilities of digital personal finance tools have evolved over the last few decades, reflecting the “hot” technology of the time. Early tools relied on rule-based decision logic and behavioral nudges (also known as gamification), such as automated savings reminders and spending alerts. Empirical research shows that these mechanisms meaningfully increase participation in financial programs and platforms and can reduce inertia in financial decision-making. More recently, as LLMs have grown, digital financial management tools have already leveraged their benefits. These include apps and services that use conversational interactions and context retention to deliver personalized guidance across a broader range of financial topics than previous services could. This has directly led to significantly higher user engagement than in previous generations of tools. However, while LLM-based systems can outperform other types of tools in personalization and engagement, they lag behind in reliability.

1.2 LLM Hallucinations

LLM hallucinations are broadly defined as instances where the output is given in a confident manner but ends up being false, inaccurate, or nonsensical. While there is a large body of evidence characterizing general hallucinations as resulting from insufficient context windows or training data [2], there is less work in financial contexts. One study, Ji et al. introduced PHANTOM, a benchmark for hallucination detection in financial long-context question answering, presented at NeurIPS 2025. Their

analysis found that the vast majority of financial hallucinations (81.3%) fall into the "misrepresentation" category, in which the model produces claims that incorrectly state financial information; contradictions and unverifiable information make up the remainder [3]. Their benchmarking results also showed that SOTA LLMs face severe challenges in detecting hallucinations in long-context financial data, reinforcing the need for external verification mechanisms, such as the one proposed in the FinVerify system, to ground the output in verifiable information.

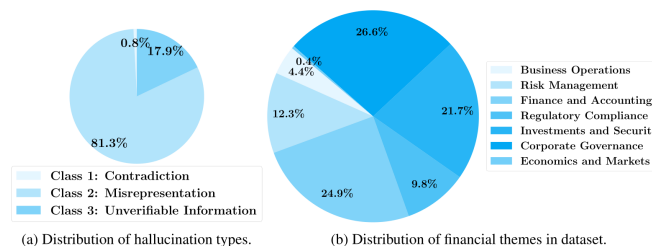


Figure 1: Distribution of hallucination types and financial themes in the PHANTOM benchmark dataset. Misrepresentation accounts for 81.3% of financial hallucinations. Adapted from Ji et al. [3].

2. Research Questions

This project is guided by three research questions, each of which will be answered in the Results section using data from our experiments.

RQ1: How frequently do LLM-based financial advice systems hallucinate when answering personal finance questions? Before we can evaluate whether a verification layer helps, we first need to understand how often the problem actually occurs. We run a baseline LLM (Qwen2.5-3B-Instruct, with no retrieval or verification) on our full question dataset and measure how often it gives incorrect or misleading answers. We break this down by question category and difficulty level to identify where hallucinations are most common.

RQ2: Does adding a RAG-based verification layer improve response accuracy compared to the baseline? This is the core question of the project. We compare the Fin-

Verify system (which grounds its answers in authoritative financial documents) against the baseline model on the same set of questions, using the same evaluation metrics. We assess whether the RAG-augmented model produces fewer hallucinations and track citation coverage to determine whether the model actually uses the sources it retrieves.

RQ3: What types of financial questions benefit most from retrieval-based verification? Not all financial questions are the same. Some are purely factual (e.g., "What is the 401(k) contribution limit for 2025?") while others are more subjective or strategic (e.g., "Should I pay off debt or build an emergency fund first?"). We break down the results by category, difficulty, and question type to understand where the verification layer adds the most value and where it has less of an impact.

To support these research questions, we also conducted an ablation study to isolate the contributions of different components of the FinVerify system. Specifically, we test configurations with and without retrieval, and experiment with different prompting strategies, to determine which parts of the pipeline are most responsible for the accuracy improvements we observe. The ablation results are reported alongside the main findings in the Results section.

3. Methodology

3.1 System Architecture

3.1.1 Baseline Implementation

For our baseline implementation, we intentionally kept it minimal to isolate later improvements. The user’s question (with multiple-choice options added to the context when applicable) is inserted in a system and user chat template and passed to the baseline model. For the baseline model, we chose Qwen2.5-3B-Instruct, as it offered a fine balance between compute and computational resources, appropriate for an end device such as a mobile phone. For the baseline, there was no retrieval, no tool use, and no post-hoc verifi-

cation step. The model answers entirely from its pre-trained memory, which means any claim depends on its training cutoff (late 2023 in this case). For multiple-choice items, we limited the output to a single letter and graded based on whether the answer matched.

For the open-ended questions, we allow up to 300 generated tokens and score with embedding cosine similarity against the reference answer (BAAI/bge-small-en-v1.5) and an LLM-as-judge rubric (claude-sonnet-4-6, scoring factuality, completeness, and advice quality on a 1–5 scale). This serves as the alternative system in our evaluation: any gain the RAG pipeline shows over this baseline must come from grounding rather than from a more capable underlying model alone.

3.1.2 FinVerify Implementation

FinVerify uses a retrieval-augmented pipeline [4] that grounds the generation of answers in a curated knowledge base of authoritative U.S. financial publications and articles. The full details of the knowledge base will be covered in the next section, but the high-level overview is that we pull these documents to form the knowledge base, so they can be chunked, sentence embeddings generated, and the results stored in a vector store. Then, at query time, the user’s question is embedded with the same sentence embedding model, and we retrieve the top $k=5$ passages with a *where* filter on the question’s topic. This ensures that the retrieval process stays within the relevant domain of the question (e.g., retirement questions will only see the chunks related to retirement documents). The retrieved passages are then appended together into a numbered context block and sent to our LLM model `claude-sonnet-4-6` with a system prompt that requires inline [n] citations for every claim the passages support. Empirical findings from testing the FinVerify system led us to relax this criterion from its original form, so we permit the system to answer well-established general personal-finance questions that the LLM model would have encoun-

tered during its pre-training phase. These include topics like the 50/30/20 rule, HMO vs. PPO, dollar-cost averaging, etc. Including specific documents that cover these general concepts in the knowledge base would have inflated the vector store’s size, slowing down FinVerify and making it less likely to be used by consumers. FinVerify’s output is bounded to 4–7 sentences (700 max tokens) and we score each answer with the same MC accuracy / cosine / LLM-judge harness as the baseline, plus a *citation coverage* metric. This metric is defined as the fraction of the k retrieved passages in which the retrieved passage actually appears in the answer. We use this metric to detect cases where retrieval succeeded, but the model ignored the supplied evidence. Figure 4 shows the full architecture diagram of the FinVerify system.

3.1.3 FinVerify Knowledge Base

FinVerify’s knowledge base is also an important component of the system, as it enables the verification layer to confidently provide accurate information. The knowledge base (KB) is a collection of 18 publicly available, verifiable U.S. personal-finance publications taken from five government and regulatory agencies. These include the IRS, Consumer Financial Protection Bureau (CFPB), SEC, Investor.gov, SSA, and HealthCare.gov. The breakdown of publishers is shown in Figure 3, with IRS publications dominating because federal tax and retirement rules carry the most impactful numerical thresholds (e.g. contribution limits, deduction phase-outs, capital-gains brackets) that LLMs are most likely to hallucinate as seen in previous research.

We built a custom pipeline to download, chunk, and ingest each source into our vector store. Each source is downloaded once by a Python script, `src/knowledge_base/ingest.py`, and split into approximately 800-character passages with 100-character overlap so that we don’t miss out any important context in the overlap. After being embedded with BAAI/bge-small-en-v1.5, each chunk

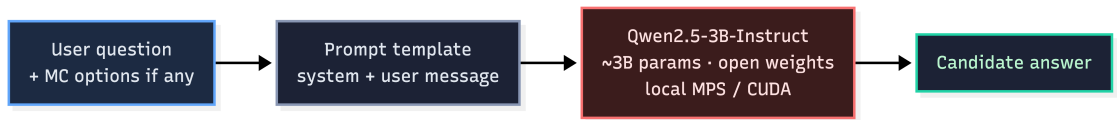


Figure 2: Baseline system architecture. The user’s question is formatted with a prompt template and passed directly to Qwen2.5-3B-Instruct running locally, with no retrieval or verification step.

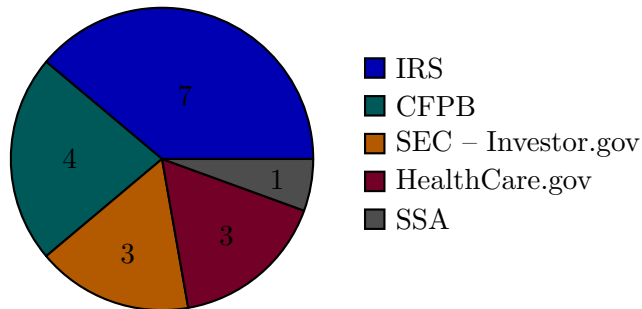


Figure 3: Composition of the FinVerify knowledge base by publisher (18 source documents total).

is persisted to a **ChromaDB** collection named `finverify_kb` along with a metadata block (`publisher`, `title`, `topic`, `source_slug`) that supports the `where` filter in order to narrow down the topics (i.e., tax, retirement, etc). We intentionally kept the corpus relatively small and stable rather than having a collection of 100+ documents. While the latter approach might improve our metrics and make them look “better”, the associated slowdown would not be justifiable for future work or commercial applications. Additionally, we wanted to ensure that all documents in the KB are verified and accurate, but validating 100+ articles would be infeasible within the scope of this project.

3.2 Evaluation Dataset

To evaluate the FinVerify system, we carefully compiled a custom dataset covering different aspects of personal finance. The evaluation dataset consists of 156 personal finance questions, divided into three subsets. Each subset has the same six topics: retirement, tax, investing, credit and debt, insurance, and budgeting. The first subset, *standard_questions* (42 items),

consists of questions about topics considered need-to-know for good personal finance literacy. These questions are answered by various documents from the IRS, FINRA, the Consumer Financial Protection Bureau (CFPB), and the Social Security Administration (SSA). This first subset is the only one that contains multiple-choice questions (30 of the 42 questions). The second subset, *open_ended_hard* (42 items), contains hand-authored questions designed to test known LLM failure modes in the personal finance context. This is where the previously conducted literature review on types of digital personal finance tools comes in handy, as it provides specific examples of questions that these tools struggle with. Examples of this include outdated annual figures, niche eligibility rules for financial programs, and common misconceptions where half-correct answers are easy to output confidently, but fail under careful examination. The third and final subset of questions, *reddit_questions* (72 items), is sampled from the public `PersonalFinance_v2` dataset [5] on Hugging Face and mapped onto the same six-topic taxonomy as previously discussed. It was important to include this subset, since the previous two subsets could feasibly have been captured during a generic LLM’s pre-training. This final subset, on the other hand, contains messy, context-dependent questions from real users facing financial challenges in the real world. Each question item has corresponding metadata for the question text, options (when applicable), reference answer, explanation, difficulty, source, URL, and topic tags. To be easily referenced and extensible, the dataset is stored as JSON, organized by subset and topic, in the project’s public GitHub repository. Table 1 shows the per-topic, per-subset distribution.

Topic	Standard		Hard	Reddit	Total
	MC	OE	OE	OE	
Budgeting	2	4	7	12	25
Credit & Debt	5	1	7	12	25
Insurance	4	2	7	12	25
Investing	5	3	7	12	27
Retirement	7	1	7	12	27
Tax	7	1	7	12	27
Total	30	12	42	72	156

Table 1: Evaluation dataset breakdown by topic, subset, and question type. MC = multiple-choice; OE = open-ended.

3.3 Evaluation Metrics

For evaluation metrics, we score every model output depending on whether it was a multiple-choice or open-ended question. **Multiple-choice accuracy** is fairly straightforward, it’s just an exact letter match (A/B/C/D) on the 30 MC items using the grader in a Python script, `src/eval/metrics.py`. For the remaining 126 open-ended questions, we calculate several metrics. One of those is **embedding cosine similarity** between the candidate and the reference answer using an embedding model `BAAI/bge-small-en-v1.5`. While this is a fairly weak indicator on its own, it is useful as a quick check of whether the reference answer is adequately represented in the candidate output. The stronger metric for open-ended questions is an **LLM-as-judge rubric**[6]: `claude-sonnet-4-6` scores each candidate against the expected answer on three qualities — factuality, completeness, and advice quality — on a 1–5 scale.

4. Results and Findings

We evaluate four system types on the 156-item dataset described previously. The systems are **Qwen-3B baseline** (open-weights, no retrieval), **Claude (no RAG)** (frontier model, no retrieval, and introduced as Ablation #1 to isolate the contribution of retrieval from the contribution of model scale), the **strict-prompt RAG variant** (an early version of

FinVerify whose system prompt explicitly said not to use knowledge it got from its pre-training), and **FinVerify** itself (the relaxed-prompt RAG configuration described in Section 3.1.2)

4.1 Overall Performance (above)

System	MC acc.	Judge 1–5	Cos 0–1	Cite cov.
Qwen-3B (no RAG)	76.7%	2.99	0.883	0.00
Claude (no RAG)	96.7%	4.75	0.901	0.00
RAG, strict prompt	70.0%	3.08	0.880	0.51
FinVerify (RAG, relaxed)	96.7%	4.58	0.898	0.33

Table 2: Headline metrics across all 156 items. MC is computed on the 30 multiple-choice items, and the remaining columns are relevant for the 126 open-ended items. Judge is the mean of factuality / completeness / advice quality on a 1–5 scale (`claude-sonnet-4-6` as judge). Citation coverage is the fraction of the $k=5$ retrieved chunks whose index appears in the answer.

4.2 Key Findings

For the strict-prompt RAG variant model, we see an expected pattern: factuality rises slightly over the Qwen baseline ($3.03 \rightarrow 3.34$) while completeness drops ($2.90 \rightarrow 2.72$). This shows that this variant fails to answer well-known concepts that any model would have encountered during pre-training, since the answer is not found word-for-word in the retrieved passages. The relaxed-prompt FinVerify configuration recovers all three qualities simultaneously (see Appendix Table 3 for the full breakdown).

When we look at how each system handles the different subsets, the Qwen baseline drops off noticeably on the Reddit questions (-1.45 vs. standard), which makes sense as these are real people asking messy, situation-specific questions that don’t have clean textbook answers. FinVerify, on the other hand, stays relatively stable across all three subsets, with less than 0.6 variation (Appendix Table 4). This is a good sign that the RAG grounding helps most with exactly the kinds of questions a real user would ask.

The score distribution further shows this gap: the Qwen baseline has 49/126 (39%) of open-

ended items scoring ≤ 2 , while both Claude (no RAG) and FinVerify eliminate this left tail entirely, with zero items below 3 (Appendix Table 6).

On the citation side, FinVerify includes an average of 1.67 passage citations per answer, and 106 out of 156 answers have at least one inline citation. Interestingly, the strict-prompt variant actually cites more often (2.54 per answer), but that doesn’t lead to better answers. This leads us to think the model was cited more aggressively without actually helping. The relaxed prompt cites less but scores higher, which tells us that more citations aren’t necessarily better if the grounding prompt isn’t well-tuned (Appendix Table 7).

4.3 Ablation Summary

Table 2 also serves as the ablation table. Here are the key findings from the ablation study:

- First, if we just swap the model from Qwen-3B to Claude while keeping everything else the same (no retrieval in either case), the judge mean jumps by +1.76 and MC accuracy goes up by +20 points. This tells us how much of the improvement comes from **model scale alone**.
- Switching to the relaxed prompt while keeping everything else the same recovers most of what the strict prompt cost: judge mean rises by +1.50 and MC accuracy returns to 96.7%. This shows the contribution of **prompt design**. It is the same model, same retrieval capability, but a much better grounding strategy.
- The overall takeaway of this ablation study is slightly disappointing, but not entirely unexpected. FinVerify matches Claude-no-RAG on MC accuracy (29/30 vs 29/30) and trails only slightly on judge mean (4.58 vs 4.75). While this seems like FinVerify has no advantage over just a better-trained LLM, we can also see that FinVerify is the only configuration out of the four that provides **verifiable**

citations (33% mean coverage, 106/156 answers with at least one inline citation). This is where relying on just the accuracy-based metrics doesn’t tell the full story, as giving users financial advice that they can actually verify is the goal of this project.

5. Discussion

5.1 Implications

The results in Section 4 speak directly to the three research questions posed in Section 2.

- **RQ1: How frequently do LLM-based financial advice systems hallucinate?** As our results show, hallucination rates depend entirely on what model is doing the answering. The baseline model used (Qwen2.5-3B-Instruct) produced answers scoring ≤ 2 on the judge rubric for 49 of 126 open-ended items (39%) and missed 7 of 30 multiple-choice items. It did the worst on real-user Reddit questions (judge mean 2.36) where patterns like outdated figures (e.g. the 2023 IRA contribution limit cited as the 2025 limit) and incorrect rules (capital-gains income taxed as ordinary income) appeared. However, a frontier-class model with no retrieval capabilities (such as Claude Sonnet 4.6) had a higher judge mean value of 4.75, showing that general personal-finance questions can now be answered by most models without needing an additional RAG layer.
- **RQ2: Does adding a RAG-based verification layer improve accuracy compared to the baseline?** Our results show that compared to the baseline model, FinVerify improves the judge mean score from 2.99 to 4.58 (+1.59) and MC accuracy from 76.7% to 96.7% (+20 pts). So, compared to our baseline model, the RAG-based verification layer does improve accuracy. When we conducted our ablation study, we observed that using the Claude - No RAG model eliminated the score differences, suggesting that, for this dataset, retrieval may not contribute to accu-

racy as much as expected. However, it is important to note that retrieval *does* contribute to verifiability: 106 of 156 FinVerify answers (68%) carry at least one citation back to a specific IRS, CFPB, SEC, SSA, or Health-Care.gov resource, while neither baseline produces any citations. If one is using these models in a domain that requires auditing or verifiability, this is the more important takeaway than focusing solely on judge-based metrics.

- **RQ3: What types of financial questions benefit most from retrieval-based verification?** The largest improvements over the Qwen model baseline are on the topics whose answers depend most on specific thresholds: *tax* (+2.14 judge mean) and *retirement* (+1.93), with insurance, budgeting, and investing all gaining 1.20–1.61. If one looks at the dataset rather than the topic area, the gain is largest on the Reddit subset of real-user questions (+1.98) and smallest on the MC-question subset (+0.89). This confirms our findings that retrieval-based grounding helps most with messy, human-phrased questions that have unclear answers or niche rules that regular models may not know from their pre-training.

5.2 Limitations

There are a few limitations worth noting. The dataset contains 156 questions across six personal-finance topics, which is enough to demonstrate that the system works but not large enough to make strong statistical claims. It’s also limited to U.S. English and federal-level rules at a single point in time — we don’t cover state tax variation, employer-specific plan rules, or the fact that many of these figures change every year.

Another big limitation is the knowledge base’s static nature — it’s built once by `ingest.py` and doesn’t update on its own. If the IRS changes contribution limits or capital-gains brackets (which happens annually), the knowledge base goes stale until someone re-runs the

ingest. So the verifiability that FinVerify provides is only as good as the most recent ingest. On the other hand, we deliberately didn’t want the knowledge base to be overloaded or overcrowded, as that would slow the system down and make it useless.

5.3 Future Work

There are a few clear next steps to continue this work. The dataset and knowledge base could be expanded to cover state-level tax rules. We could add questions in other languages and scripts besides English to our dataset. The ingest pipeline could be set up to re-embed the latest documents when agencies publish updates. Most importantly, our main claim, that retrieval’s real contribution is verifiability, not accuracy, is ultimately a claim about how users behave, so the most valuable follow-up would be a user study to measure whether people actually trust cited answers more and whether citations help them catch hallucinations they would have otherwise missed.

6. Deliverables

All code, data, and documentation are in a public GitHub repository:

<https://github.com/niksharma99/COMS6156FinalProject>

To reproduce: clone the repo, `pip install -r requirements.txt`, set `ANTHROPIC_API_KEY`, run `python src/evaluate.py`. Full instructions in README.

7. Self-Evaluation

7.1 What I Accomplished

I built the full FinVerify system end-to-end. This includes a 156-question evaluation dataset comprising multiple subsets: an MC-question dataset I compiled from various finance agencies and 72 real-user questions sampled from a public Reddit dataset on HuggingFace. I also put together a knowledge base of 18 authoritative source documents from agencies such as

the IRS, CFPB, and SEC, and a Python system that automatically downloads, chunks, and ingests the data into a ChromaDB vector store of sentence embeddings. We have multiple models as part of the deliverable, from the Qwen2.5-3B-Instruct baseline to the RAG-augmented FinVerify system built on top of Claude Sonnet 4.6. On top of the two main systems, I ran two ablations: Claude with no retrieval and the strict-prompt RAG variant, which together let me decompose the FinVerify gain into what worked and what didn't. I also built a Vite + React demo site with various tabs (architecture, dataset, live demo, comparison, metrics) that pulls real data from the results JSONs at build time.

7.2 What I Found Challenging

The biggest research takeaway was that ablations aren't just a box you check — they're how you figure out what's actually going on. Without the Claude-no-RAG ablation, I would have written up FinVerify's +1.59 judge gain over Qwen as "RAG works." But the data actually shows that retrieval adds almost nothing on top of a capable enough model, and FinVerify's real contribution is verifiable citations (33% coverage vs. 0% for either baseline).

7.3 What I Learned

The biggest research takeaway was that ablations aren't just a box you check — they're how you figure out what's actually going on. Without the Claude-no-RAG ablation, I would have written up FinVerify's +1.59 judge gain over Qwen as "RAG works." But the data actually shows that retrieval adds almost nothing on top of a capable enough model, and FinVerify's real contribution is verifiable citations (33% coverage vs. 0% for either baseline).

References

- [1] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
- [2] L. Huang et al., "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *arXiv preprint arXiv:2311.05232*, 2023.
- [3] L. Ji, D. Seyler, G. Kaur, M. Hegde, K. Dasgupta, and B. Xiang, "PHANTOM: A benchmark for hallucination detection in financial long-context QA," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [4] Y. Gao et al., "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2024.
- [5] A. Theerthala, *PersonalFinance_v2*, 2024. [Online]. Available: https://huggingface.co/datasets/Akhil-Theerthala/PersonalFinance_v2.
- [6] L. Zheng et al., "Judging LLM-as-a-judge with MT-bench and chatbot arena," *arXiv preprint arXiv:2306.05685*, 2023.

A. Appendix

A.1 FinVerify System Architecture Diagram

A.2 Judge Rubric Breakdown

System	Fact.	Comp.	Advice	Mean
Qwen-3B (no RAG)	3.03	2.90	3.04	2.99
Claude (no RAG)	4.76	4.57	4.91	4.75
RAG, strict prompt	3.34	2.72	3.17	3.08
FinVerify	4.63	4.37	4.75	4.58

Table 3: LLM-judge per-axis means on the 126 open-ended items.

A.3 Performance by Dataset Subset

System	Standard (n=12)	Hard OE (n=42)	Reddit (n=72)
Qwen-3B (no RAG)	3.94	3.81	2.36
Claude (no RAG)	4.92	4.97	4.59
RAG, strict prompt	2.39	3.42	3.00
FinVerify	4.83	4.94	4.34

Table 4: Judge mean (1–5) on open-ended items, broken down by dataset subset.

A.4 MC Accuracy by Topic

Topic (n)	Qwen	Claude	Strict	FV
Budgeting (2)	2/2	2/2	0/2	2/2
Credit & Debt (5)	4/5	5/5	5/5	5/5
Insurance (4)	2/4	3/4	2/4	4/4
Investing (5)	4/5	5/5	4/5	5/5
Retirement (7)	4/7	7/7	6/7	6/7
Tax (7)	7/7	7/7	4/7	7/7
Total (30)	23/30	29/30	21/30	29/30

Table 5: MC accuracy by topic.

A.5 Score Distribution

A.6 Citation Behavior

System	~ 1	~ 2	~ 3	~ 4	~ 5
Qwen-3B (no RAG)	2	47	37	27	13
Claude (no RAG)	0	0	0	25	101
RAG, strict prompt	6	36	44	19	21
FinVerify	0	0	3	39	84

Table 6: Per-item judge-mean distribution across the 126 open-ended items, bucketed by rounded mean score.

System	Cov.	Cites/ans	≥ 1 cite
RAG, strict prompt	0.51	2.54	121/156
FinVerify	0.33	1.67	106/156

Table 7: Citation behavior on the 156 items.



Figure 4: FinVerify retrieval-augmented architecture. Top band: one-time knowledge base ingestion. Middle band: per-query retrieval against the ChromaDB collection with topic-based filtering. Bottom band: prompt assembly and generation with `claude-sonnet-4-6`. The dotted edge marks the only crossing of runtime boundaries.